

Git, GitHub, GitLab & VS Code

Complete Practical & Conceptual Guide

Git Core: Local Version Control & Branching

GitHub: Remote Workflow & Collaboration

GitLab: DevOps & CI/CD Pipelines

VS Code: GUI Source Control & Live Demo

CLASSROOM LEARNING EDITION

Git Foundations: Why Version Control?

Module 1

Understanding the Need for Version Control Systems (VCS)

Old Way (Without Git)

- Multiple zip files and confusing folder names:
`Project_v1.zip` , `Project_final.zip` ,
`Project_FINAL_2.zip` .
- High risk of overwriting teammate's code accidentally.
- No way to track who made what change and when.
- Hard to recover code if something breaks in production.

Smart Way (With Git)

- **Complete History:** Tracks every single change line-by-line with exact timestamps and author details.
- **Easy Rollback:** Instantly revert back to any working state if code crashes.
- **Parallel Collaboration:** Multiple developers work on different features simultaneously without conflict.

Git Architecture & Daily Commands

Module 1

The 3 Local Stages and Everyday Workflow

The 3 Local Stages of Git

- **1. Working Directory:** Local folder where files are created and modified.
- **2. Staging Area (Index):** Temporary holding area for changes selected for the next commit.
- **3. Local Repository (.git):** Permanent local database storing code snapshots.

Essential Commands

- `git init` : Turn current folder into a Git repository.
- `git status` : Check modified, staged, or untracked files.
- `git add .` : Move all changes to Staging Area.
- `git commit -m "message"` : Save staged code permanently in local history.
- `git log` : View commit history timeline.

Git Branching, Merging & Stashing

Managing Independent Workflows & Temporary Work

Module 1

1. Branching

- Isolated environment to work on new features without touching stable code.
- `git branch` : List branches.
- `git switch -c feat` : Create & switch to new branch.

2. Merging

- Combine changes from feature branch back into main branch.
- `git merge feat` : Merge `feat` into active branch.
- **Merge Conflict:** Resolved manually when same line is modified differently.

3. Stashing

- Temporarily shelve uncommitted work to switch tasks quickly.
- `git stash` : Save & clean working directory.
- `git stash pop` : Restore stashed changes back.

GitHub Platform & Remote Workflow

Module 2

Connecting Local Repositories to Cloud Backup

Git vs GitHub

- **Git:** Local command-line tool running on your laptop.
- **GitHub:** Cloud-based hosting service for Git repositories with a graphical web UI.

4 Steps Remote Workflow

- **1. Create Cloud Repo:** Make a new empty repository on GitHub.com.
- **2. Link Remote:** Connect local repo to cloud:

```
git remote add origin <URL>
```
- **3. Push Code:** Upload local commits to cloud:

```
git push -u origin main
```
- **4. Pull Updates:** Download latest team changes:

```
git pull origin main
```

GitHub Collaboration & Teamwork

Forks, Pull Requests (PRs), and Project Management

Module 2

Fork & Clone

- **Fork:** Copy someone else's repository into your GitHub account.
- **Clone:** Download cloud repository to local machine:
`git clone <URL>`

Issues & Boards

- Track bugs, feature requests, and tasks seamlessly.
- Use Kanban Boards (To Do, In Progress, Done) for team agile management.

Pull Requests (PR)

- Propose changes from your feature branch to the original project.
- Peer code review & automated checks.
- Safe merge approval process.

GitHub Deployment & Security

Static Web Hosting and Repository Protection

Module 2

GitHub Pages (Free Hosting)

- Host HTML, CSS, and JS web applications directly from your GitHub repository for free.
- **Setup Steps:**
 1. Go to Settings → Pages.
 2. Select Source: `main` branch.
 3. Click Save.
- Live URL format: `https://username.github.io/reponame`

Branch Protection Rules

- Protect `main` branch from accidental deletion or untested code.
- Require Pull Request reviews before merging.
- Require status checks/tests to pass first.
- Prevent direct force pushing.

GitLab Platform & DevOps Overview

Complete All-in-One Lifecycle & GitHub Comparison

What is GitLab?

GitLab is a complete DevOps Platform provided as a single application. It covers planning, repository management, security testing, and automated deployment.

Feature	GitHub	GitLab
Primary Focus	Code Collaboration & Open Source	Complete DevOps & CI/CD Lifecycle
Code Review Request	Pull Request (PR)	Merge Request (MR)
Hosting Model	SaaS Cloud (Mainly)	Cloud & Self-Hosted Enterprise Servers

GitLab CI/CD Pipeline Architecture

Continuous Integration & Continuous Deployment Explained

Module 3

What is CI/CD?

- **Continuous Integration (CI):** Automatically builds and tests code every time a developer pushes changes.
- **Continuous Deployment (CD):** Automatically deploys tested code to live staging/production servers.

Automated 4-Stage Pipeline Flow

1. BUILD

Compile code and assemble dependencies.

2. TEST

Run automated unit and integration tests.

3. SECURITY

Scan for vulnerabilities and secret leaks.

4. DEPLOY

Publish live application to cloud servers.

GitLab CI/CD Configuration

Controlling Automated Pipelines via YAML Scripts

Module 3

Sample `.gitlab-ci.yml` Script

```
stages:
  - build
  - test
  - deploy

build-job:
  stage: build
  script:
    - echo "Compiling project code..."

test-job:
  stage: test
  script:
    - echo "Running automated tests..."

deploy-job:
  stage: deploy
  script:
    - echo "Deploying code to live server!"
```

Key Elements

- **Stages:** Defines the sequence of execution pipeline.
- **Jobs:** Specific tasks executed under each stage.
- **Runners:** Lightweight background agent programs that run these scripts on remote servers.

VS Code Integration (Graphical Source Control)

Module 4

Managing Git visually without complex terminal commands

Source Control Tab (`ctrl+shift+g`)

- **Stage Changes:** Click the `+` button next to modified files.
- **Commit:** Type commit message in box and click Checkmark (✓).
- **Sync / Push:** Single-click button to sync changes with cloud repo.

Top Productivity Tools

- **Integrated Terminal (`ctrl+``):** Run terminal commands without leaving code editor.
- **GitLens Extension:** Shows line-by-line blame, author, and commit history directly in editor.
- **3-Way Merge Editor:** Visually resolve merge conflicts with side-by-side comparison.

Classroom Demo Checklist

Complete Step-by-Step Practical Demonstration

Module 4

Step 1: Local Setup & File Creation

```
# 1. Initialize repository
mkdir student-demo && cd student-demo
git init

# 2. Create index.html and commit
git add .
git commit -m "Initial HTML setup"
```

Step 2: Connect Remote & Push

```
# 3. Connect to GitHub/GitLab
git remote add origin <URL>

# 4. Push code to main branch
git push -u origin main

# 5. Branch demo
git switch -c feature-btn
git push origin feature-btn
```

Summary

Students can now track local code, collaborate via cloud branches, automate deployments, and manage everything inside VS Code!